

```
> claude "собери бота"  
> /start → тема → имя  
> → контакт → заявка  
> 4 теста ✓
```

ALOVLAB · ПРОФЕССИОНАЛЬНЫЙ PLAYBOOK

Работа с Claude над Telegram-ботом

Не «напиши код». Полная дисциплина: сценарий, состояния, секреты, тесты, деплой. Разбор на реальном примере — бот записи на консультацию.

Мастер-промпт целиком

Диаграмма состояний

4 обязательных теста

СОДЕРЖАНИЕ

Маршрут playbook

Двадцать шагов: от постановки задачи до бота, который работает 24/7 и не роняет твой токен в чужие руки.

01	Что ты соберёшь	03
02	Модель: кто кому отвечает	04
03	Сценарий: одно действие на шаг	05
04	Диаграмма состояний	06
05	Мастер-промт целиком	07
06	Анатомия хорошего промта	08
07	Что должен отдать Claude	09
08	Секреты: токен и .env	10
09	Регистрация у BotFather	11
10	Локальный запуск	12
11	4 обязательных теста	13
12	Обработка сбоев	14
13	Плохой промт vs хороший	15
14	Хостинг: 24/7	16
15	Честность в AI-инженерии	17
16	Ошибки и фиксы	18
17	Сделай сейчас + проверка	19
18	Дальше — на курсе	20
19	Путь в команду AlovLab	21
20	Контакты	22

ШАГ 01

Что ты соберёшь

Бота, который принимает заявки на консультацию: выбор темы, имя, контакт, заявка администратору. С обработкой ошибок и без единой утечки токена.

**ЧТО ЭТО****Проект, не файл**

bot.py + requirements.txt +
.env.example + .gitignore +
README.

ДЛЯ ЧЕГО**Приём заявок**

Диалог за 4 шага, заявка
приходит тебе лично в
Telegram.

ДИСЦИПЛИНА**Как у разработчика**

Секреты вне кода, тесты
перед показом, честный
README.

ЧТО ДОЛЖНО ПОЛУЧИТЬСЯ

Рабочий прототип: /start запускает диалог, каждый шаг ждёт ответ, в конце заявка
приходит администратору. Готов к переносу на хостинг.

ШАГ 02

Модель: кто кому отвечает

У бота четыре звена. Claude пишет только одно из них — код. Остальное решаешь ты.

ЗВЕНО	КТО ЭТО	ЧТО ДЕЛАЕТ
Человек	твой клиент	пишет /start, отвечает на вопросы бота
Telegram	платформа	доставляет сообщения между человеком и твоим кодом
Твой код	bot.py	ведёт диалог по сценарию, хранит состояние
Админ	ты	получает готовую заявку личным сообщением

Важно. Если код не запущен — диалог обрывается на полпути, и человек застревает. Поэтому сначала проверяешь запуск и только потом показываешь бота другим.

Вывод: сначала схема (кто кому что говорит), потом генерация кода. Не наоборот.

ШАГ 03

Одно действие на шаг

«Умный бот, который всё понимает» — плохая постановка задачи. Хорошая — точный сценарий.

- 1 /start.** Бот здоровается и предлагает выбрать одну из трёх тем консультации.
- 2 Тема.** Человек выбирает кнопкой. Бот запоминает выбор и просит имя.
- 3 Имя → контакт.** Два простых вопроса подряд, каждый ждёт текстовый ответ.
- 4 Подтверждение → заявка.** Бот показывает итог, затем отправляет тебе личное сообщение.

ПРАВИЛО

- ☐ Каждый шаг должен иметь ожидаемый ответ. Если человек прислал не то — бот переспрашивает, а не ломается.
- ☐ /cancel работает на любом шаге и прерывает сценарий чисто, без зависшего состояния.

ШАГ 04

Диаграмма состояний

Нарисуй это до генерации кода — так сразу видно дыры в сценарии.

СОСТОЯНИЕ	ЖДЁТ	ДАЛЬШЕ
/start	команду запуска	→ ТЕМА
ТЕМА	нажатие кнопки	→ ИМЯ
ИМЯ	текст (не пустой)	→ КОНТАКТ
КОНТАКТ	текст или номер	→ ЗАЯВКА → АДМИНУ

Каждая строка таблицы — это то, что ты отдаёшь Claude как техническое задание, а не как смутное пожелание «сделай удобно».

ЗАЧЕМ ЭТО НУЖНО

Если позже бот работает не так — ты сразу видишь, в каком состоянии он застрял, и просишь Claude починить конкретный переход, а не «почини всё».

ШАГ 05

Мастер-промт целиком

Это не шаблон с [СКОБКАМИ] — это цельное техническое задание. Меняй тему бота, структуру оставляй.

МАСТЕР-ПРОМПТ · СКОПИРОВАТЬ

СКОПИРОВАТЬ

Собери проект Telegram-бота на Python для приёма заявок на консультацию.

Сценарий: `/start` → выбор одной из трёх тем → имя → контакт → подтверждение → приватное сообщение с заявкой администратору. На каждом шаге работает `/cancel`. Обработай пустой ввод, неожиданный тип сообщения, повторный `/start` и сбой отправки администратору.

Токен и ID администратора считывай из переменных окружения. Не записывай токен в код, логи или Git. Выдай `bot.py`, `requirements.txt`, `.env.example` без секретов, `.gitignore` и README с командами запуска для Windows и macOS/Linux. Для локального прототипа используй `long polling`, не включай одновременно `webhook`.

Покажи четыре ручных теста: `/start`, пустой ответ, `/cancel`, получение заявки администратором. Не заявляй об успешной интеграционной проверке без реального токена. Отдельно опиши публикацию на хостинге: переменные окружения, перезапуск процесса, мониторинг ошибок. Сначала покажи план файлов и сценарий, потом код.

Обрати внимание на порядок требований: сначала сценарий и файлы, потом код, потом честные тесты, потом хостинг. Такой порядок сам по себе дисциплинирует Claude не торопиться с ответом.

ШАГ 06

Проси не код. Проси проект.

Шесть требований, которые спасают от переделок — разбор мастер-промпта по частям.

№	ТРЕБОВАНИЕ	ЗАЧЕМ
01	Цель	Claude знает, что вообще делает бот — приём заявок, не абстракция.
02	Путь	start → контакт: чёткая последовательность шагов диалога.
03	Сбои	пустой ввод, /cancel: код не падает на реальных людях.
04	Секреты	из окружения: токен не утечёт в код или Git.
05	Запуск	команды по шагам для двух систем — ты не гадаешь.
06	Проверка	5 сценариев (включая хостинг) — бот проверен, не просто написан.

Убери любой из шести пунктов — и получишь код, который выглядит рабочим, но подведёт на первом реальном пользователе.

ШАГ 07

Что должен отдать Claude

Не один кусок кода, а запускаемый комплект. Если инструкции запуска нет — проект ещё не готов.

```
bot.py # сценарий и обработка сообщений
requirements.txt # зависимости проекта
.env.example # поля для секретов, без значений
.gitignore # защита секретов от случайного коммита
README.md # команды запуска Windows + macOS/Linux
```

ЧТО ВСТАВИТЬ

Ничего — это результат мастер-промпта, не отдельный запрос.

ЧТО ПОЛУЧИТЬ

Пять файлов, которые вместе образуют рабочий проект, готовый к git init.

Если Claude прислал только код без README и .env.example — попроси прямо: «дай остальные файлы проекта из требований».

ШАГ 08

Токен = ключ от бота

Токен даёт полный доступ к боту от твоего имени. Обращайся с ним как с паролем.

ГДЕ	ТОКЕН
В коде (bot.py)	НЕТ — никогда не пиши токен прямой строкой
В .env (не в Git)	ДА — локальный файл с реальным токеном
В .env.example (в Git)	НЕТ — только имя переменной без значения
В общем чате или скриншоте	НИКОГДА

ЕСЛИ ТОКЕН ВСЁ ЖЕ УТЁК

- 👉 Зайди к @BotFather, команда /revoke — старый токен умирает, получаешь новый.
- 👉 Обнови .env новым токеном, перезапусти бота.

.gitignore из мастер-промпта как раз существует, чтобы .env физически не мог попасть в Git, даже если ты забудешь об этом в спешке.

ШАГ 09

Регистрация у @BotFather

Официальный способ Telegram создать бота. Занимает минуту.

- 1 **Найди @BotFather** в Telegram — официальный бот с синей галочкой.
- 2 **Команда /newbot** — задай имя и юзернейм, который должен заканчиваться на «bot».
- 3 **Получи токен** — длинная строка цифр и букв. Это и есть твой ключ доступа.
- 4 **Узнай свой Telegram ID** (для переменной администратора) — спроси у любого бота вроде @userinfobot.
- 5 **Впиши оба значения в .env**, не в .env.example и не в код.

ШАГ 10

Локальный запуск

Для прототипа — long polling. Не включай одновременно webhook, это конфликтует.

POLLING (ДЛЯ ТЕСТА)

Бот сам спрашивает Telegram: «есть новые сообщения?». Просто, работает на любом компьютере, не нужен публичный адрес.

ВЕБНООК (ДЛЯ ПРОДАКШНА)

Telegram сам стучится к боту по адресу. Нужен публичный сервер с HTTPS. Разбираем на курсе.

- 1** Установи зависимости из requirements.txt командой из README.
- 2** Запусти bot.py — командой из README для своей системы.
- 3** Напиши /start своему боту в Telegram и пройди сценарий целиком.

ШАГ 11

«Работает» надо доказать

Пройди путь клиента. И сломай его специально — не только удачный сценарий.

ВВОД	ОЖИДАЕМЫЙ РЕЗУЛЬТАТ
<code>/start</code>	открылся первый шаг, показаны кнопки тем
<code>пусто</code>	бот просит повторить, диалог не ломается
<code>/cancel</code>	сценарий остановлен, состояние сброшено
<code>заявка</code>	админ получил её личным сообщением

ПРАВИЛО

- ☐ Если хотя бы один тест красный — верни Claude точное описание, что пошло не так, а не общее «не работает».
- ☐ Тестируй с реального телефона, не только с компьютера — там другая клавиатура и другой темп ввода.

ШАГ 12

Обработка сбоев

Реальные люди присылают боту не то, что ты ожидал. Четыре сценария из мастер-промпта.

Пустой ввод. Человек нажал «отправить», не написав текст. Бот вежливо просит повторить, не роняется.

Неожиданный тип сообщения. Прислали фото вместо текста. Бот объясняет, что ждёт, и не теряет прогресс.

Повторный /start. Человек передумал и начал заново. Старое состояние сбрасывается чисто, без дублей.

Сбой отправки администратору. Например, ты заблокировал бота. Клиент должен увидеть, что заявка не потерялась молча.

Если Claude не покрыл один из этих случаев — прямо укажи на него, ссылаясь на этот список.

ШАГ 13

Плохой промпт против хорошего

Разница не в длине, а в точности.

ПЛОХО

«Сделай бота для записи на консультацию.»

Нет сценария, нет требований к секретам, нет тестов. Получишь код, который выглядит рабочим и подводит на первом же пустом сообщении.

ХОРОШО

Мастер-промпт со страницы 07: точный сценарий по шагам, требования к секретам, список тестов, требование честности про непроверенную интеграцию.

Хороший промпт для инженерной задачи — это техническое задание, а не пожелание. Ты не обязан уметь программировать, но обязан уметь точно описать, что должно происходить на каждом шаге.

ШАГ 14

Чтобы бот работал 24/7

Локальный запуск живёт, пока открыт терминал. Для постоянной работы — хостинг.

ЛОКАЛЬНО

Для теста. Закрыв терминал — бот замолчал. Быстро проверить сценарий.

НА ХОСТИНГЕ

Работает 24/7, есть мониторинг ошибок. Для реального использования.

- 1 Спроси Claude:** «назови 2–3 простых способа разместить Python-бота на long polling, дай инструкцию для новичка».
- 2 Задай переменные окружения** на хостинге (токен, ID администратора) — так же, как в .env, но через панель сервиса.
- 3 Настрой автоперезапуск процесса** — если бот упадёт из-за сети, он должен подняться сам.
- 4 Проверь мониторинг ошибок** — чтобы узнать о сбое раньше, чем это заметит клиент.

ШАГ 15

Честность в AI-инженерии

Мастер-пром프트 неспроста требует: не заявлять об успешной проверке без реального токена.

ТАК НЕЛЬЗЯ

- ✗ Просить Claude «подтвердить», что интеграция протестирована, если токен ещё не вставлен.
- ✗ Публиковать бота без прогона всех 4 тестов вручную.
- ✗ Молча пропускать случаи из раздела «Обработка сбоев», потому что «и так сойдёт».

ТАК ПРАВИЛЬНО

- ✔ Тестируешь с реальным токеном сам, своими руками, перед показом другим.
- ✔ Если Claude написал «должно работать» — это гипотеза, не факт. Проверяешь и только потом веришь.
- ✔ Честно пишешь в README, что это прототип, если это прототип.

Это тот же принцип, что и во всей нашей работе: ИИ ускоряет, а не подтверждает результат вместо тебя.

ШАГ 16

Частые ошибки и фиксы

Пять граблей, которые встречаются чаще всего.

Забыл .gitignore, токен попал в историю Git. Фикс: /revoke у BotFather, новый токен, .gitignore с самого начала следующего проекта.

Webhook и polling включены одновременно. Фикс: выключи webhook (setWebhook с пустым url) перед локальным polling.

Бот теряет состояние между сообщениями. Фикс: попроси Claude явно показать, где и как хранится состояние диалога.

README без команд для твоей ОС. Фикс: прямо укажи систему — Windows, macOS или Linux — и попроси команды под неё.

Тесты не проводились, но бот «вроде работает». Фикс: пройди все 4 теста со страницы 13 прежде, чем показывать бота другим.

ШАГ 17

Сделай сейчас

За один вечер пройди путь от задачи до работающего прототипа.

- 1** Опиши свои **4 шага диалога** по образцу диаграммы со страницы 06 — под свою задачу, не обязательно консультации.
- 2** Собери **мастер-промпт** по формуле со страницы 07–08 и отправь Claude.
- 3** Запусти **локально** и пройди диалог сам целиком.
- 4** Пройди **4 проверки** со страницы 13, прежде чем показать кому-то ещё.

ПРОВЕРЬ ПЕРЕД ПОКАЗОМ ДРУГИМ

- ☐ Токен нигде не виден в коде, логах или Git.
- ☐ Все 4 теста пройдены вручную, не только «по ощущениям».
- ☐ README объясняет запуск понятно даже не программисту.
- ☐ Заявка реально доходит до администратора.

ДАЛЬШЕ

Один бот — это один инструмент. А есть система

Ты прошёл дисциплину постановки задачи для ИИ. На курсе применяешь её ко всему контент-конвейеру.

Сценарий, диаграмма состояний, честные тесты — этот метод работает не только для ботов. Курс «Нейросети и ChatGPT для каждого» учит собирать систему целиком: тексты, визуал, видео, звук, аватары — и то, что ты только что прошёл, боты и автоматизацию.

НА КУРСЕ

Webhook + продакшн

Как перенести бота с polling на настоящий сервер.

НА КУРСЕ

Приём оплаты

Как подключить к боту простую оплату.

НА КУРСЕ

Конвейер

Бот, сайт, контент и автоматизация как одна система.

СЕЙЧАС СКИДКА 50% НА ТАРИФ ПРО

49 990 ₽ вместо 99 990 ₽, до 30 сентября. Начать — alovlab.ru.

ПУТЬ ДАЛЬШЕ

Путь в команду AlovLab

Если начало получаться и хочется делать это для других — есть куда расти.

Собираешь ботов с такой дисциплиной?

Сильные ученики AlovLab заходят в реальные проекты: собирают ботов и автоматизацию под задачи брендов, набивают портфолио на живых кейсах и растут рядом с командой.

Telegram-боты

Автоматизация

AI-ассистенты

Контент-конвейеры

Это работа и практика, а не обещание трудоустройства. Но дорога открыта: покажи, что доводишь результат до проверенного продакшна.

Бизнесу, которому нужен бот или автоматизация под ключ — это AlovLab Studio.

Отправляешь бриф, сборку и конвейер берём на себя.



ALOVLAB · АВТОКОНТЕНТ

Один бот — инструмент. Система — это курс.

Скидка 50% на тариф ПРО до 30 сентября — 49 990 ₽
вместо 99 990 ₽. Нужен бот под бизнес под ключ —
отправь бриф в студию.

Начать курс · alovlab.ru

Бриф студии · [@alovlab](https://t.me/alovlab)

Telegram · t.me/AlovLab